
EEE5776. Applied Machine Learning



Module 2. Supervised Learning and Experimental Design

Bagging

Boosting - AdaBoost and Gradient Boosting Machines

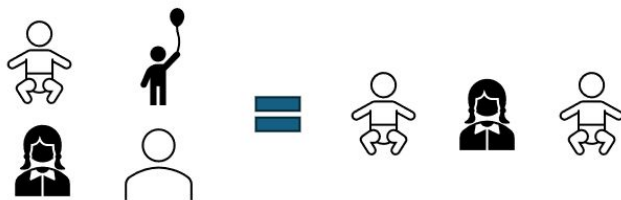
A series of white lines on the left side of the slide, consisting of several horizontal lines that are offset to the right at regular intervals, creating a stepped or staircase-like effect.

Bagging- Recap

Pasting

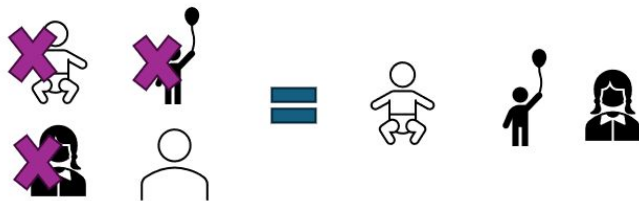
Bagging vs. Pasting

Sampling with replacement



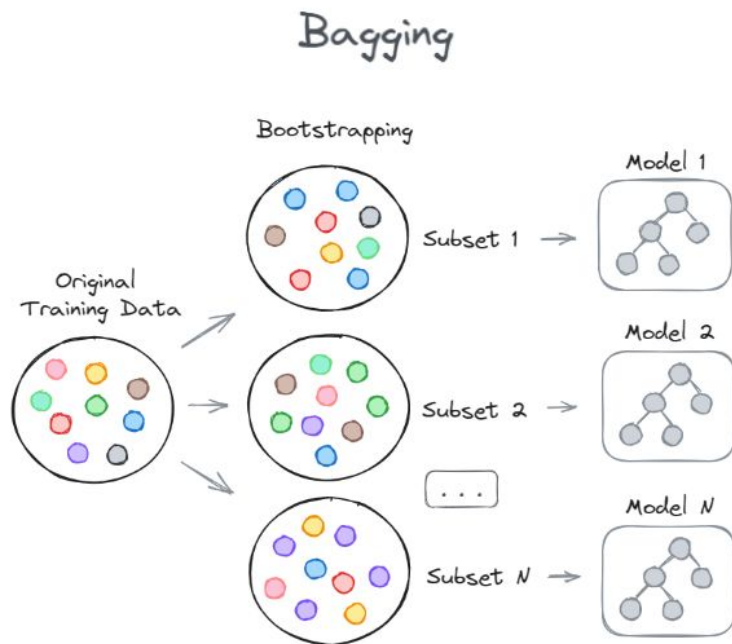
Bootstrap Aggregating (bagging)

Sampling without replacement



Pasting

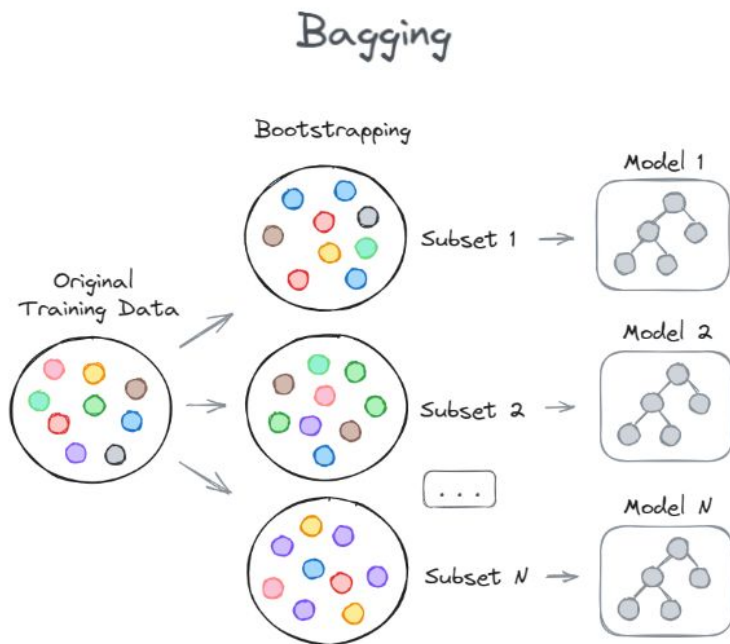
Why?



Parallel processing

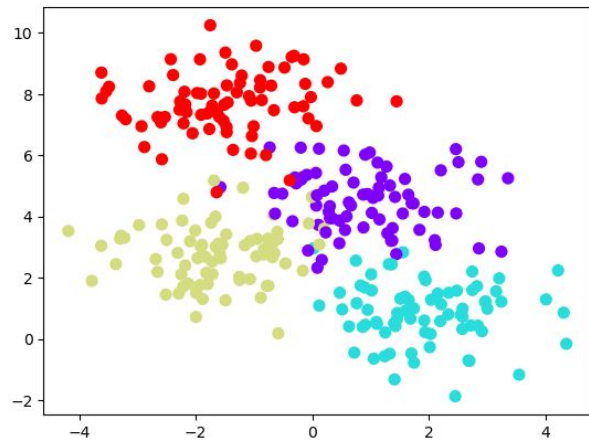
Once all predictors are trained, the **ensemble** can make a prediction for a new instance by simply **aggregating** the predictions of all predictors. The aggregation function is typically the **statistical mode**, i.e., the most frequent prediction, just like a **hard voting classifier for classification**, or the **average for regression**.

Why?



- Each individual predictor has a **higher bias** (i.e. it is underfitting) than if it were trained on the original training set, but **aggregation reduces both bias and variance.**
- Generally, the net result is that the ensemble has a similar bias but a lower variance than a single predictor trained on the original training set.

Decision Tree Example



```
## EXAMPLE: DECISION TREE CLASSIFIER

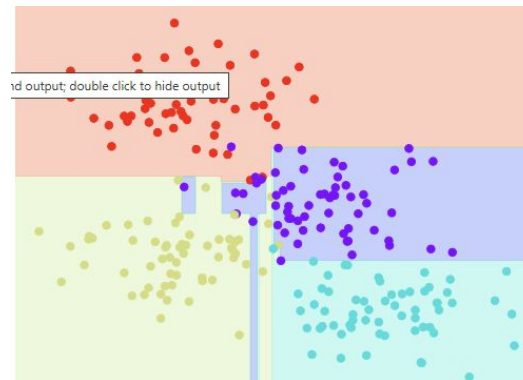
# Setup pipeline
dec_tree = Pipeline([['scaler', MinMaxScaler()],
                     ('dec_tree', DecisionTreeClassifier(random_state=0))])

# Train pipeline
dec_tree.fit(X_train, t_train)

# Visualize decision surface
visualize_classifier(dec_tree, X_train, t_train)

# Predictions & Metric Scores
y_train1 = dec_tree.predict(X_train)
y_test1 = dec_tree.predict(X_test)
print('Training performance:',
      accuracy_score(t_train, y_train1))
print('Test performance:',
      accuracy_score(t_test, y_test1))
```

Training performance: 1.0
Test performance: 0.9



Decision Trees are known as
weak learners

Bagging - Decision Tree Example

```
## EXAMPLE: BAGGING WITH DECISION TREES
```

```
# Setup pipeline
```

```
bag1 = Pipeline([('scaler', MinMaxScaler()),
                  ('bagging', BaggingClassifier(DecisionTreeClassifier(max_depth=5, random_state=0),
                                             n_estimators=100,
                                             bootstrap=True,
                                             oob_score=True,
                                             max_samples=0.8, # resamples 80% of training samples using Bootstrap
                                             random_state=1))])
```

```
# Train pipeline
```

```
bag1.fit(X_train, t_train)
```

```
# Visualize decision surface
```

```
visualize_classifier(bag1, X_train, t_train)
```

```
# Predictions & Metric Scores
```

```
y_train1 = bag1.predict(X_train)
```

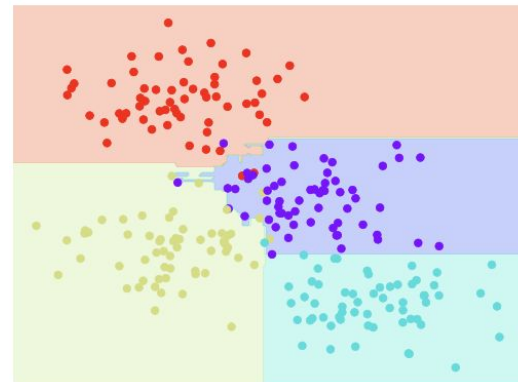
```
y_test1 = bag1.predict(X_test)
```

```
print('Training performance:',
      accuracy_score(t_train, y_train1))
```

```
print('Test performance:',
      accuracy_score(t_test, y_test1))
```

```
Training performance: 0.9791666666666666
```

```
Test performance: 0.9166666666666666
```



Bagging - K-Nearest Example

```
## EXAMPLE: BAGGING WITH K-NEAREST NEIGHBORS CLASSIFIER
from sklearn.neighbors import KNeighborsClassifier

# Setup pipeline
bag2 = Pipeline([('scaler', MinMaxScaler()),
                  ('bagging', BaggingClassifier(KNeighborsClassifier(n_neighbors=5),
                                                n_estimators=100,
                                                bootstrap=True,
                                                oob_score=True,
                                                max_samples=0.8,
                                                random_state=1))])

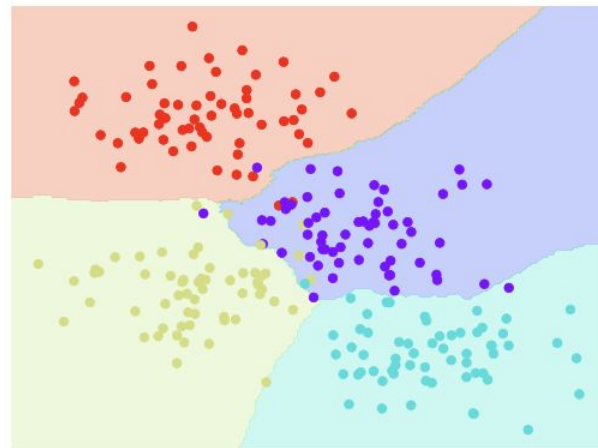
# Train pipeline
bag2.fit(X_train, t_train)

# Visualize decision surface
visualize_classifier(bag2, X_train, t_train)

# Predictions & Metric Scores
y_train2 = bag2.predict(X_train)
y_test2 = bag2.predict(X_test)
print('Training performance:',
      accuracy_score(t_train, y_train2))
print('Test performance:',
      accuracy_score(t_test, y_test2))
```

Training performance: 0.9458333333333333

Test performance: 0.9166666666666666



Out-of-Bag Evaluation

- With bagging, some instances may be sampled several times for any given predictor, while others may not be sampled at all.
- By default a BaggingClassifier samples m training instances with replacement (`bootstrap=True`), where m is the size of the training set.
- A bagging ensemble can be evaluated using oob instances, without the need for a separate validation set: indeed, if there are enough estimators, then each instance in the training set will likely be an oob instance of several estimators, so these estimators can be used to make a fair ensemble prediction for that instance. Once you have a prediction for each instance, you can compute the ensemble's prediction accuracy (or any other metric).

```
bag.oob_score_
```

```
0.9083333333333333
```

```
y_test = bag.predict(X_test)
```

```
from sklearn.metrics import accuracy_score
```

```
accuracy_score(t_test, y_test)
```

```
0.9666666666666667
```

Out-of-Bag Evaluation

- The oob decision function for each training instance is also available through the `oob_decision_function_` variable.
- In this case (since the base estimator has a `predict_proba()` method), the decision function returns the class probabilities for each training instance.
- For example, the oob evaluation estimates that the fourth training instance has a 79.07% probability of belonging to class 4 (and 20.93% of belonging to class 1, 0% to class 2 and 0% to class 3):

```
bag1.named_steps.bagging.oob_decision_function_

# Matrix of shape N-by-C, N is the number of samples and C is the number of classes

array([[0.00000000e+00, 1.00000000e+00, 0.00000000e+00, 0.00000000e+00],
       [7.76397516e-04, 0.00000000e+00, 9.99223602e-01, 0.00000000e+00],
       [0.00000000e+00, 0.00000000e+00, 1.00000000e+00, 0.00000000e+00],
       [2.09302326e-01, 0.00000000e+00, 0.00000000e+00, 7.90697674e-01],
```

Random Patches and Random Subspaces

- The `BaggingClassifier` class supports sampling the features as well.
- Sampling is controlled by two hyperparameters: `max_features` and `bootstrap_features`.
- They work the same way as `max_samples` and `bootstrap`, but for feature sampling instead of instance sampling.
- Thus, each predictor will be trained on a random subset of the input features. This technique is particularly useful when you are dealing with high-dimensional inputs (such as images).
- Sampling both training instances and features is called the **Random Patches** method.
- Keeping all training instances (by setting `bootstrap=False` and `max_samples=1.0`) but sampling features (by setting `bootstrap_features=True` and/or `max_features` to a value equal or smaller than 1.0) is called the **Random Subspaces** method.

Random Patches - Example

```
## EXAMPLE: RANDOM PATCHES

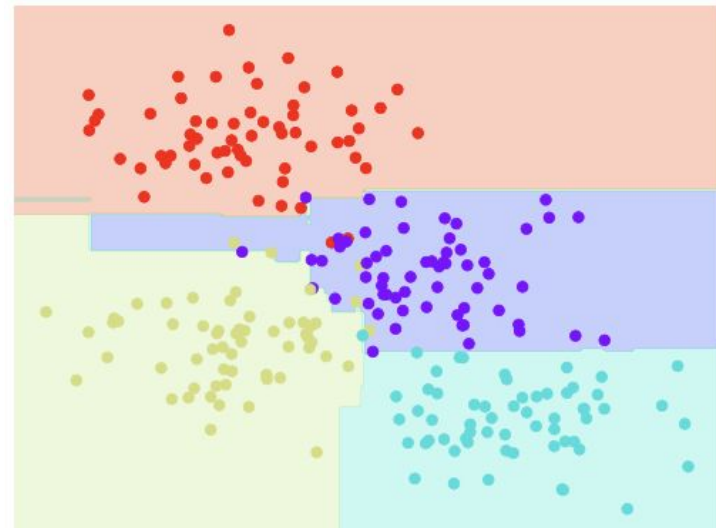
# Setup pipeline
rnd_patch = Pipeline([('scaler', MinMaxScaler()),
                      ('bagging', BaggingClassifier(DecisionTreeClassifier(max_depth=3, random_state=0),
                                                  n_estimators=60,
                                                  bootstrap=True,
                                                  max_samples=0.8,
                                                  bootstrap_features=True,
                                                  max_features=1.0,
                                                  random_state=0)))]

# Train pipeline
rnd_patch.fit(X_train, t_train)

# Visualize decision surface
visualize_classifier(rnd_patch, X_train, t_train)

# Predictions & Metric Scores
y_train3 = rnd_patch.predict(X_train)
y_test3 = rnd_patch.predict(X_test)
print('Training performance:',
      accuracy_score(t_train, y_train3))
print('Test performance:',
      accuracy_score(t_test, y_test3))
```

```
Training performance: 0.9708333333333333
Test performance: 0.8833333333333333
```



Random Spaces - Example

```
## EXAMPLE: RANDOM SUBSPACES

# Setup pipeline
rnd_subspace = Pipeline([('scaler', MinMaxScaler()),
                          ('bagging', BaggingClassifier(DecisionTreeClassifier(max_depth=2, random_state=0),
                                                       n_estimators=100,
                                                       bootstrap=False, # Resampling WITHOUT replacement
                                                       max_samples=1.0,
                                                       bootstrap_features=True,
                                                       max_features=1.0,
                                                       random_state=0)))]

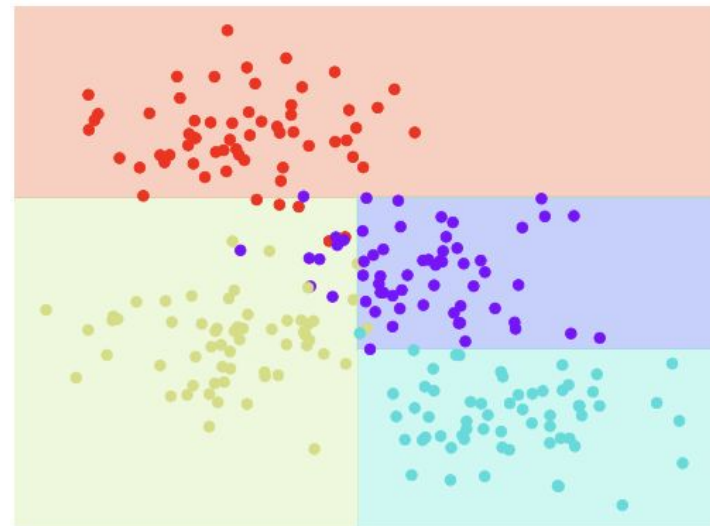
# Train pipeline
rnd_subspace.fit(X_train, t_train)

# Visualize decision surface
visualize_classifier(rnd_subspace, X_train, t_train)

# Predictions & Metric Scores
y_train4 = rnd_patch.predict(X_train)
y_test4 = rnd_patch.predict(X_test)
print('Training performance:',
      accuracy_score(t_train, y_train4))
print('Test performance:',
      accuracy_score(t_test, y_test4))
```

Training performance: 0.9708333333333333

Test performance: 0.8833333333333333



A series of white, stylized lines on the left side of the slide, resembling a staircase or a series of parallel paths that recede into the distance.

Random Forests-Recap

Random Forests

An ensemble of randomized decision trees is known as a random forest.

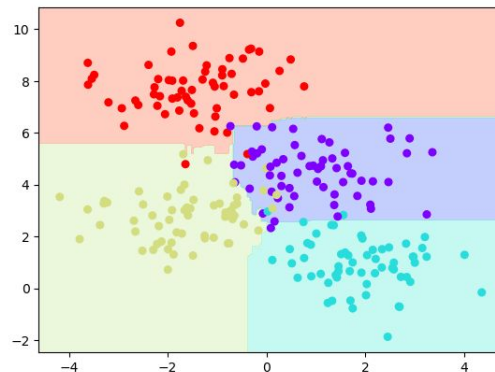
In Scikit-Learn, such an optimized ensemble of randomized decision trees is implemented in the `RandomForestClassifier` estimator, which takes care of all the randomization automatically. All you need to do is select a number of estimators, and it will very quickly (in parallel, if desired) fit the ensemble of trees:

```
from sklearn.ensemble import RandomForestClassifier

RandomForestClassifier?

model = RandomForestClassifier(n_estimators=100,
                              max_depth=4,
                              random_state=0)

visualize_classifier(model, X_train, t_train);
```



We see that by averaging over 100 randomly perturbed models, we end up with an overall model that is much closer to our intuition about how the parameter space should be split.

```
model.feature_importances_
array([0.35408312, 0.64591688])
```

Extremely Randomized Trees (or Extra-Trees)

- When you are growing a tree in a Random Forest, at each node only a random subset of the features is considered for splitting (as discussed earlier).
- It is possible to make trees even more random by also using random thresholds for each feature rather than searching for the best possible thresholds (like regular Decision Trees do).
- A forest of such extremely random trees is called an Extremely Randomized Trees ensemble (or Extra-Trees for short).

This technique trades more bias for a lower variance. It also makes Extra-Trees much faster to train than regular Random Forests, because finding the best possible threshold for each feature at every node is one of the most time-consuming tasks of growing a tree.

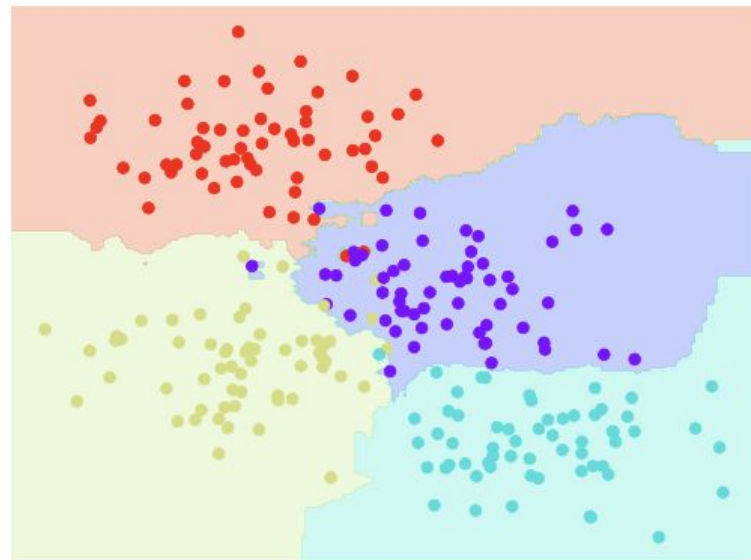
Extremely Randomized Trees (or Extra-Trees)

```
from sklearn.ensemble import ExtraTreesClassifier
```

```
ExtraTreesClassifier?
```

```
model2 = ExtraTreesClassifier(n_estimators=100,  
                             bootstrap=True,  
                             oob_score=True,  
                             max_samples=0.8,  
                             random_state=1)
```

```
visualize_classifier(model2, X_train, t_train);
```



Feature Importance

Another great quality of Random Forests is that they make it easy to measure the relative importance of each feature. Scikit-Learn measures a **feature's importance** by looking at how much the tree nodes that use that feature reduce impurity on average (across all trees in the forest).

Iris Dataset Example

```
# Load Iris Dataset ( 4 features)
from sklearn.datasets import load_iris
iris = load_iris()

# Setting up pipeline
rnf_clf = Pipeline([('scaler', MinMaxScaler()),
                    ('rnf_clf', RandomForestClassifier(n_estimators=500))])

# Train pipeline
rnf_clf.fit(iris["data"], iris["target"])

# View feature importances
for name, score in zip(iris["feature_names"],
                      rnf_clf.named_steps['rnf_clf'].feature_importances_):
    print(name, score)
```

```
sepal length (cm) 0.09948572985900493
sepal width (cm) 0.02194323192541615
petal length (cm) 0.4459714851236513
petal width (cm) 0.4325995530919275
```

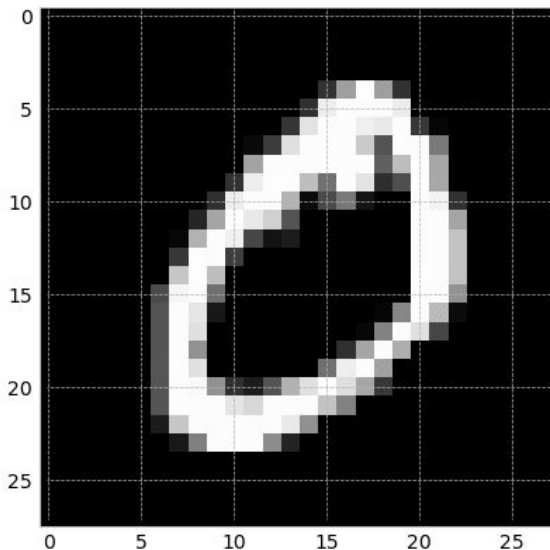
Feature Importance

```
from sklearn.datasets import fetch_openml
# image dataset, 28x28 grayscale, 784 dimensions
mnist = fetch_openml('mnist_784', version=1, as_frame=False, parser='auto')
X, t = mnist.data, mnist.target
t = t.astype(np.uint8)

# Partition into training and test sebytes
X_train, t_train, = X[:10000], t[:10000] #fetch a small sample size for training for demo purposes

plt.imshow(X[1,:].reshape(28,28), cmap='gray');
```

MNIST Example



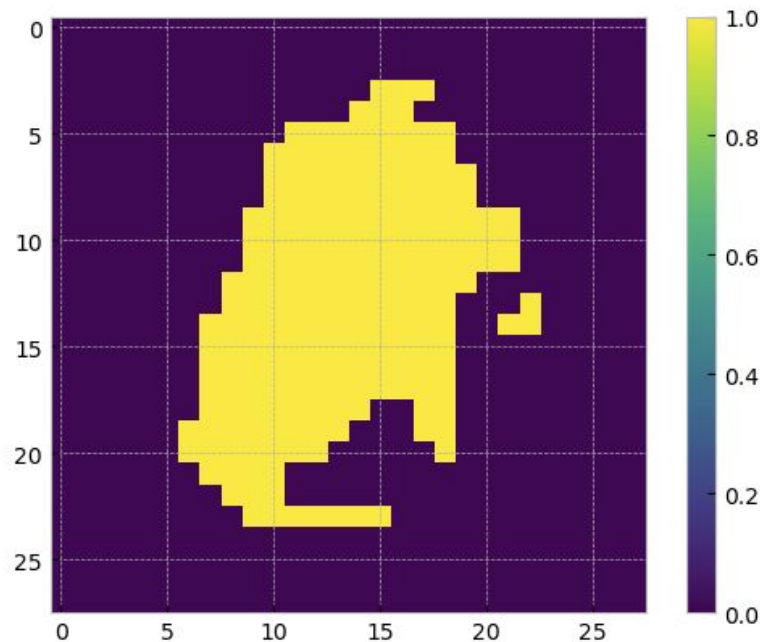
Feature Importance



```
mask = 1*(model.named_steps.rnf_clf.feature_importances_>0.002)
```

```
plt.imshow(mask.reshape(28,28))  
plt.colorbar();
```

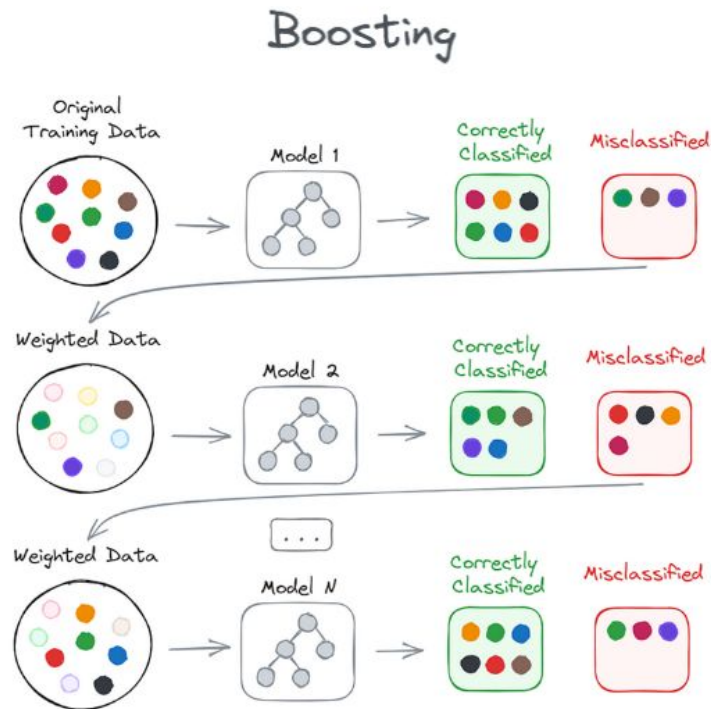
MNIST Example





Boosting

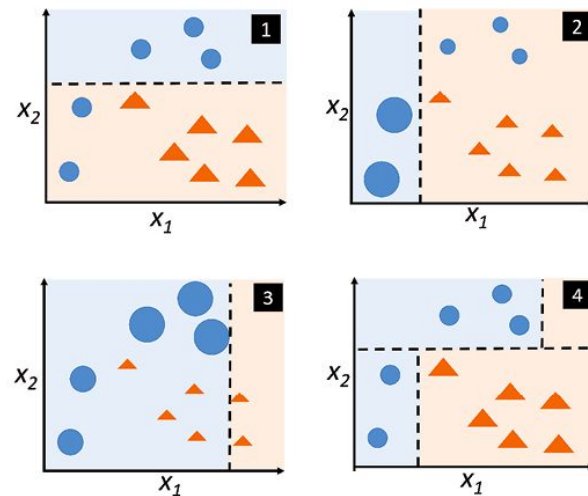
Boosting



Boosting (or Hypothesis Boosting) refers to any Ensemble method that can combine several weak learners into a strong learner. The general idea of most boosting methods is to **train predictors sequentially**, each trying to correct its predecessor. There are many boosting methods available, but by far the most popular are **AdaBoost (short for Adaptive Boosting)** and **Gradient Boosting**.

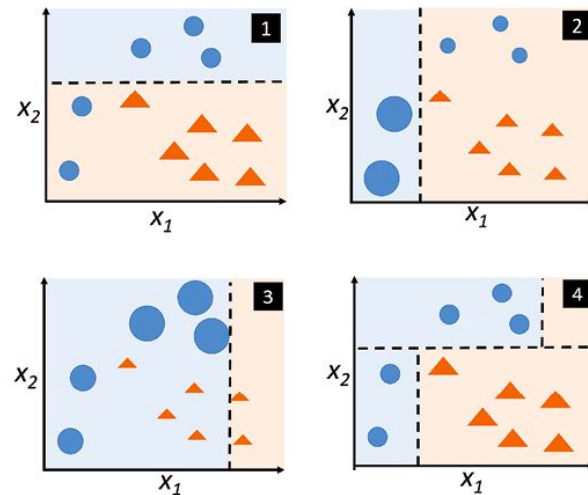
AdaBoost or Adaptive Boosting

- It builds many small models (like tiny decision trees).
- Each new model pays extra attention to the mistakes made by previous models.
- It tries to fix errors by focusing more on the hard-to-classify examples.
- Works well for simple problems but can struggle with noisy data.



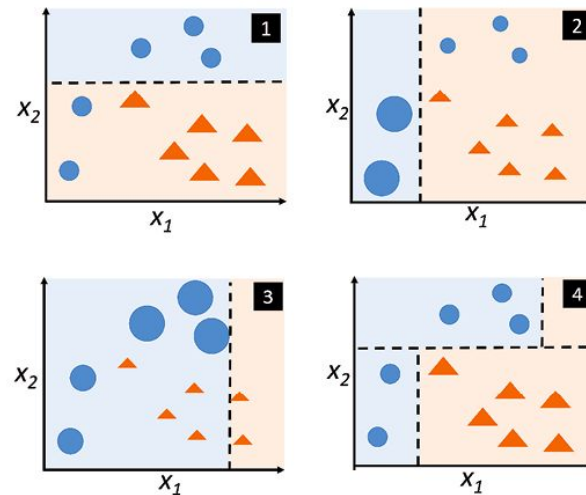
AdaBoost or Adaptive Boosting

- Classifier 1 (clf1) classifies the points. There are some points (blue points) that get misclassified.
- The misclassified points are given more weights and weights of correctly classified points get reduced.



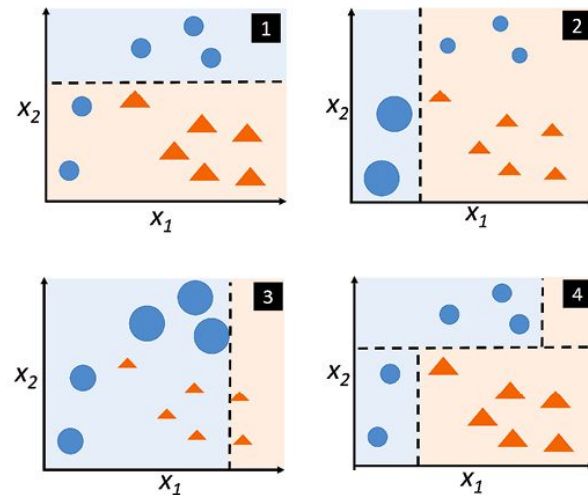
AdaBoost or Adaptive Boosting

- A new classifier (clf2) is trained with new training data set having more weights assigned to the misclassified points and lesser weights to the correctly classified points (thus, adaptive resampling). Look at pic 2 where misclassified points have given higher weight which is represented using larger circle and correctly classified points got reduced in size.
- New classifier (clf2) again results in misclassification of few points (3 blue points).
- The misclassified points classified by clf2 gets higher weights (larger circle) and correctly classified points get lower weights (size reduced).



AdaBoost or Adaptive Boosting

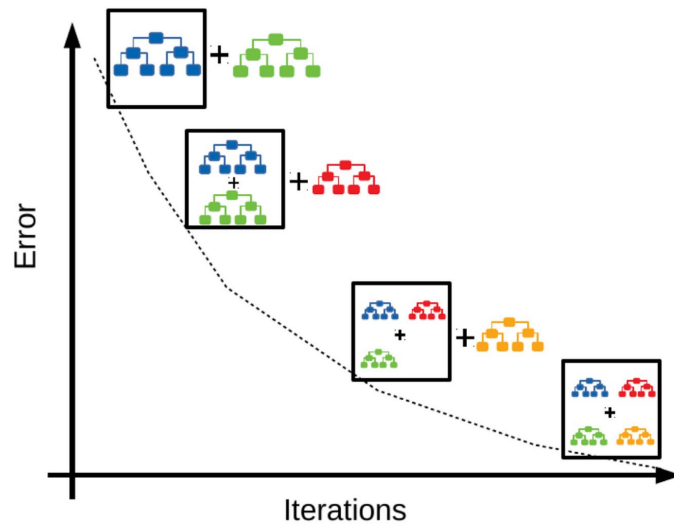
- Third classifier (clf3) gets retrained with new training datasets with weights for each data points updated. Take a look at picture 3. Finally, the ensemble adaptive boosting classifier (clf4) is constructed by ensembling three classifiers constructed / fitted / trained using different training datasets created as a result of adaptive resampling.
- Take a look at picture 4 which represents the adaptive boosting (AdaBoost) classifier created by ensembling three classifiers.



Math Foundation and Implementation included in the Notebook supporting this class

Gradient Boosting

- It also builds many small models step by step.
- Instead of focusing on mistakes, it tries to reduce the overall error by improving on what the last model got wrong.
- Uses a smarter approach (gradient descent) to adjust and improve the predictions. More powerful and flexible but needs more tuning.



Implementation included in the Notebook supporting this class